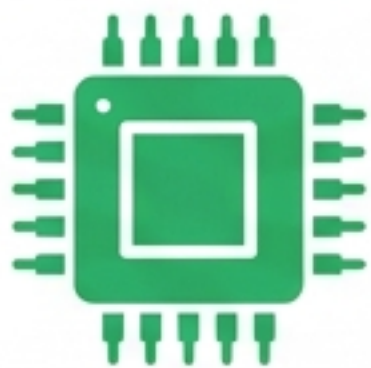


การเชื่อมต่อ Raspberry Pi กับ ESP32 และเซ็นเซอร์สภาพอากาศผ่าน Wi-Fi

อบรมเชิงปฏิบัติการ Internet of Things (ส่วนที่ 3) | บรรยายโดย ผศ. ดร. ชนันทกรณ์ จันแดง



เชื่อมต่อ ESP32 กับ
Raspberry Pi
และอ่านข้อมูลจาก
เซ็นเซอร์

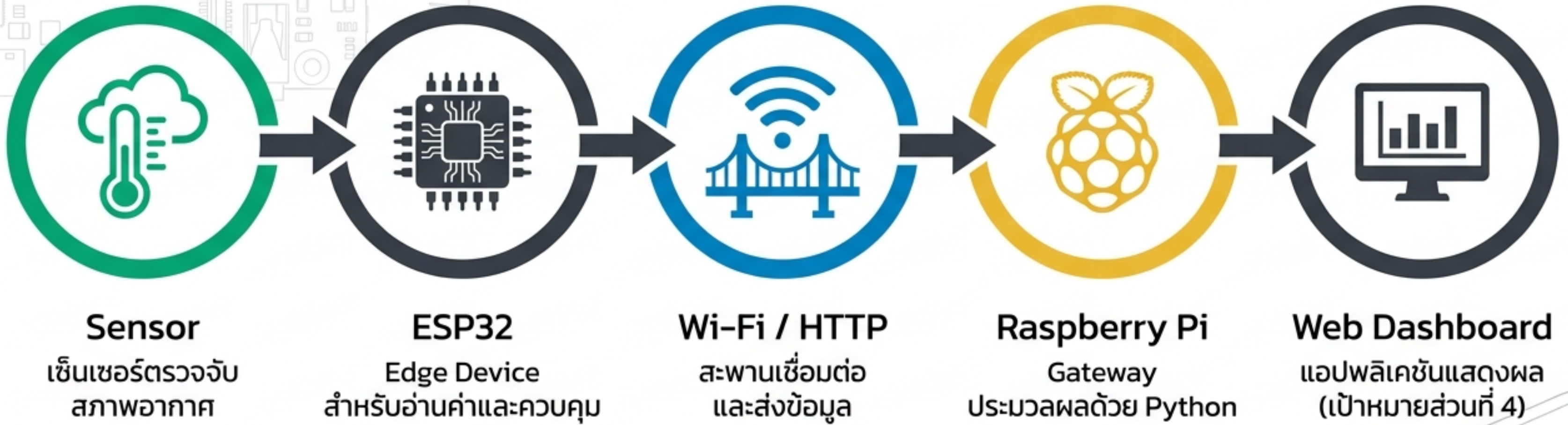


ส่งและรับข้อมูล
ระหว่างอุปกรณ์ผ่าน
เครือข่าย Wi-Fi
([HTTP/JSON](#))



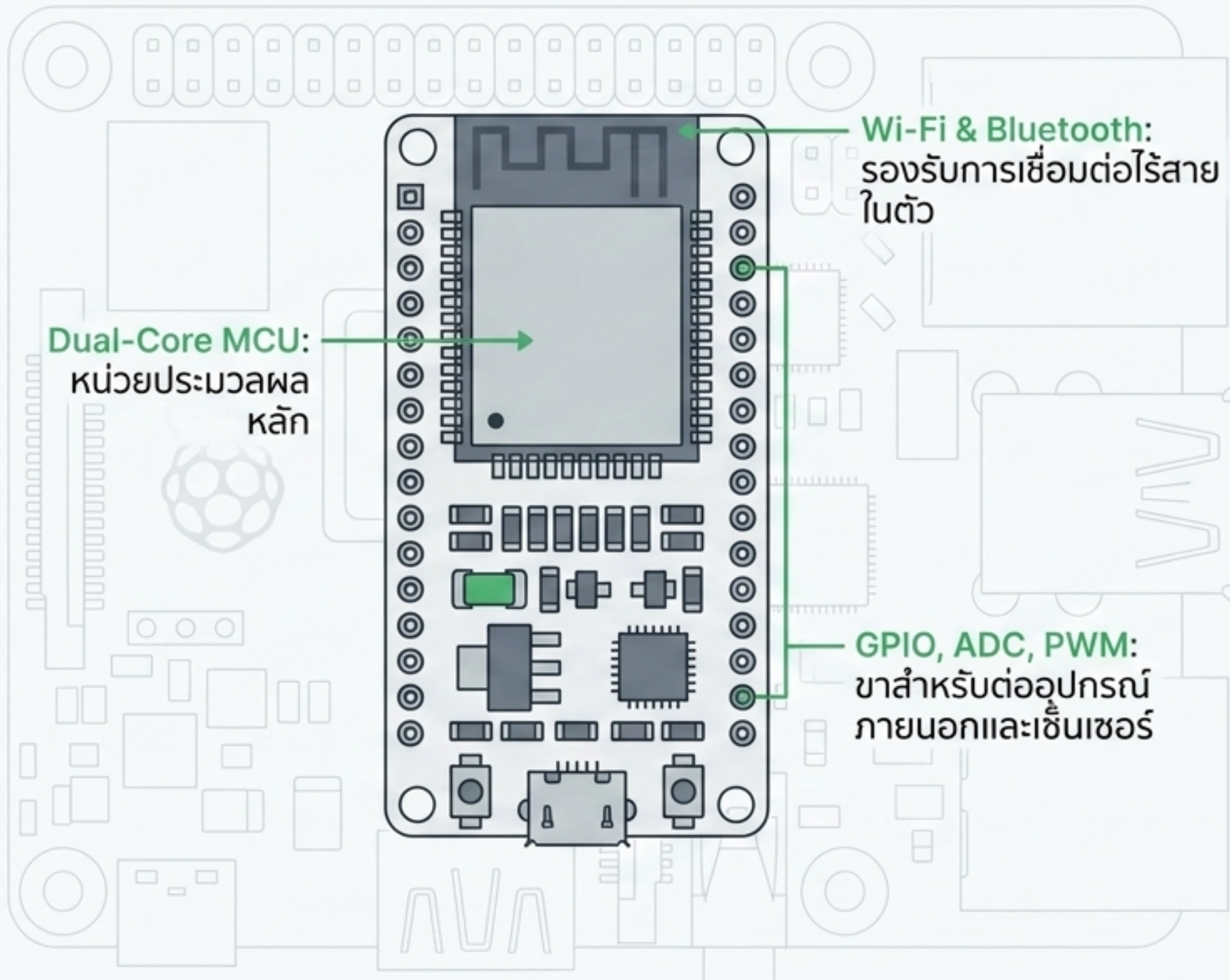
ใช้ Python ประมวลผล
และบันทึกข้อมูลเพื่อ
เตรียมสร้าง Web
Dashboard

เส้นทางการไหลของข้อมูลจากโลกกายภาพสู่หน้าจอแสดงผล



การอบรมในส่วนที่ 3 จะโฟกัสที่การสร้างและเชื่อมต่อ 4 องค์ประกอบแรกให้ทำงานร่วมกันได้อย่างสมบูรณ์

ESP32 คือหัวใจสำคัญในการเก็บข้อมูลที่จุดปฏิบัติงาน (Edge Device)



เหมาะสำหรับระบบ:



Smart Home (บ้านอัจฉริยะ)



Smart Farm (ฟาร์มอัจฉริยะ)



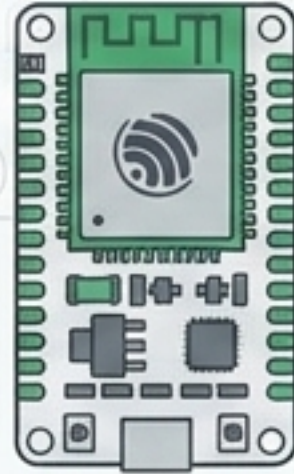
Sensor Node (จุดเก็บข้อมูล)



IoT Device ขนาดเล็กที่ต้องการประหยัดพลังงาน

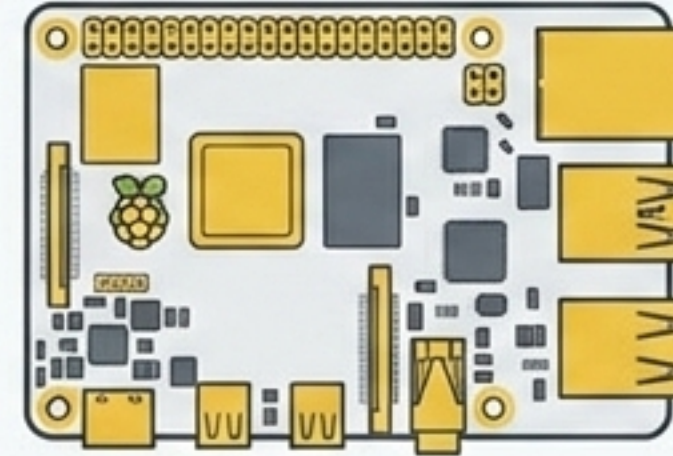
การแบ่งหน้าที่ระหว่าง Microcontroller และ Linux Computer

ESP32 (Sensor Controller)



ประเภท:	Microcontroller
ภาษาที่ใช้:	Arduino / C++
บทบาท:	"มือและตา" - เชื่อมต่อเซ็นเซอร์, ดึงข้อมูลจากสภาพแวดล้อม
จุดเด่น:	ทำงานแบบ Real-time, กินไฟต่ำ, บุตรระบบทันที

Raspberry Pi (Data Processor)



ประเภท:	Linux Computer
ภาษาที่ใช้:	Python
บทบาท:	"สมอง" - รับข้อมูล, ประมวลผล, บันทึกไฟล์, ทำหน้าที่เป็น Web Server
จุดเด่น:	พลังประมวลผลสูง, รันระบบปฏิบัติการเต็มรูปแบบ

สรุป: ESP32 ทำหน้าที่เก็บข้อมูลจากเซ็นเซอร์ (Data Collection)
ส่วน Raspberry Pi ทำหน้าที่ประมวลผลและจัดการข้อมูล (Processing)

การเตรียมเครื่องมือและทดสอบบอร์ดด้วย Arduino IDE



1. ติดตั้ง Arduino IDE



2. เพิ่ม URL สำหรับ ESP32 Board Manager



3. เลือกบอร์ดและ COM Port ให้ตรงกับอุปกรณ์



4. กด Upload เพื่อส่งโปรแกรมลงบอร์ด

Workshop Action

เป้าหมาย: ทดลอง Upload โปรแกรม Blink LED เพื่อยืนยันการเชื่อมต่อ



หากไฟ LED บนบอร์ดกระพริบตามจังหวะ แสดงว่าคอมพิวเตอร์สามารถสื่อสารกับ ESP32 ได้สมบูรณ์ พร้อมสำหรับการเขียนโปรแกรมขั้นต่อไป

การเชื่อมต่อ ESP32 เข้ากับเครือข่าย Wi-Fi (Station Mode)



- **Wi-Fi Station Mode:** ESP32 ทำตัวเป็น Client เพื่อเกาะสัญญาณเครือข่ายเดียวกับ Raspberry Pi
- **ตัวแปรสำคัญ:** **SSID** (ชื่อเครือข่าย) และ **Password** (รหัสผ่าน)
- **DHCP:** ระบบจะจ่าย IP Address ให้ ESP32 อัตโนมัติเมื่อเชื่อมต่อสำเร็จ

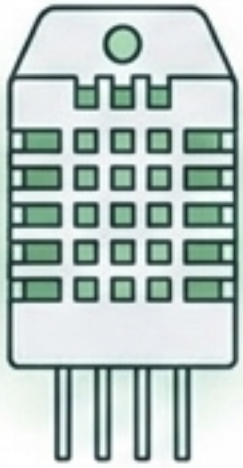
Workshop Action Box

เป้าหมาย: เขียนโปรแกรมเชื่อมต่อ Wi-Fi และดึง IP Address

Serial Monitor

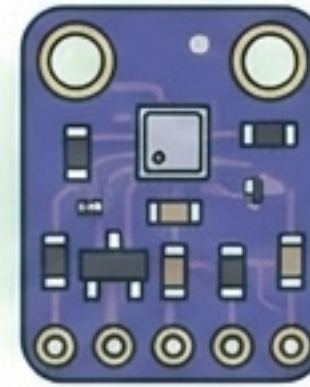
```
Connecting to WiFi...  
Connected to WiFi  
IP Address: 192.168.1.45
```


การเลือกใช้และต่อวงจรเซ็นเซอร์สภาพอากาศ



DHT22 (Temperature & Humidity)

- วัดอุณหภูมิ (Temperature)
- วัดความชื้นสัมพัทธ์ (Humidity)
- เหมาะสำหรับการวัดสภาพอากาศพื้นฐานในร่มและกลางแจ้ง



BME280 (Temperature, Humidity, Pressure)

- วัดอุณหภูมิ (Temperature)
- วัดความชื้น (Humidity)
- วัดความกดอากาศ (Pressure)
- ให้ความแม่นยำสูงกว่าและครอบคลุมข้อมูลมากกว่า



Workshop Demo: สาธิตการต่อวงจรเซ็นเซอร์เข้ากับขา GPIO ของ ESP32 อย่างถูกต้องก่อนเริ่มเขียนโปรแกรม

การอ่านค่าสภาพแวดล้อมเข้าสู่ระบบดิจิทัล

Workshop Steps

- 1. ติดตั้ง Library สำหรับ DHT22 หรือ BME280 ใน Arduino IDE
- 2. เขียนคำสั่งเรียกอ่านค่าจากเซ็นเซอร์
- 3. พิมพ์ผลลัพธ์ออกทาง Serial Monitor

Code-to-Reality

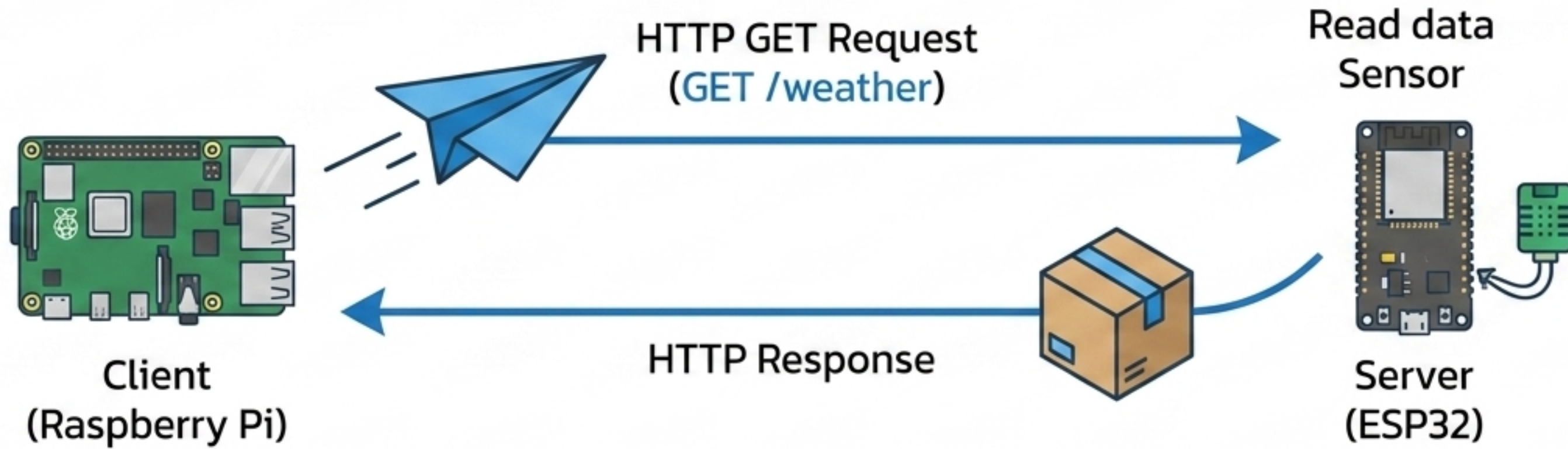
```
float temp = dht.readTemperature();
```

Simulated Output Box

Temperature : 30.4 °C
Humidity : 71 %

ขณะนี้ ESP32 สามารถรับรู้สภาพอากาศในโลกกายภาพได้อย่างสมบูรณ์แล้ว

สถาปัตยกรรมการสื่อสารผ่าน HTTP และ REST API



HTTP: โพรโทคอลมาตรฐานที่ใช้ในการสื่อสารบนเว็บ

REST API: ช่องทางที่ออกแบบให้โปรแกรม (Pi) สามารถสั่งงานและขอข้อมูลจากอีกโปรแกรม (ESP32) ได้อย่างเป็นระบบ

JSON: รูปแบบแฟ้มเก็บข้อมูลสากลที่ใช้สำหรับแลกเปลี่ยนข้อมูล

การแพ็กข้อมูลด้วย JSON ผ่าน ESP32 Web Server

การสร้าง URL Endpoint บน ESP32: `http://[ESP32_IP]/weather`

```
{  
  "temperature": 30.4,  
  "humidity": 71  
}
```

'Key' (String) : 'Value' (Float)

ข้อมูลถูกจัดเรียงเป็นคู่
อ่านง่ายทั้งคนและเครื่อง

ภารกิจ

เปิด Browser บนคอมพิวเตอร์หรือ
โทรศัพท์มือถือที่อยู่ใน Wi-Fi เดียวกัน

เป้าหมาย

พิมพ์ IP ของ ESP32 ต่อด้วย /weather เพื่อดู
โครงสร้างข้อมูล JSON ที่ ESP32 ส่งออกมา

Raspberry Pi รับข้อมูลข้ามเครือข่ายด้วย Python

Core Concept: การใช้ Python Library `requests` ในการทำหน้าที่เป็น Client เพื่อดึงข้อมูลอัตโนมัติ



ส่งคำขอ (GET)

```
response = requests.get('http://ESP32_IP/weather')
```

รับและแปลง (Parse)

```
data = response.json()
```

Workshop: เขียนสคริปต์ Python บน Raspberry Pi และสั่งรันโปรแกรมเพื่อดึงข้อมูลข้าม Wi-Fi มาแสดงผลใน Terminal เพื่อยืนยันว่าการเชื่อมต่อทั้งระบบทำงานประสานกัน

การแยกส่วนข้อมูล (Parsing JSON) เพื่อนำไปใช้งาน

```
data = {'temperature': 30.4, 'humidity': 71}
```

```
temp = data['temperature']
```

```
hum = data['humidity']
```

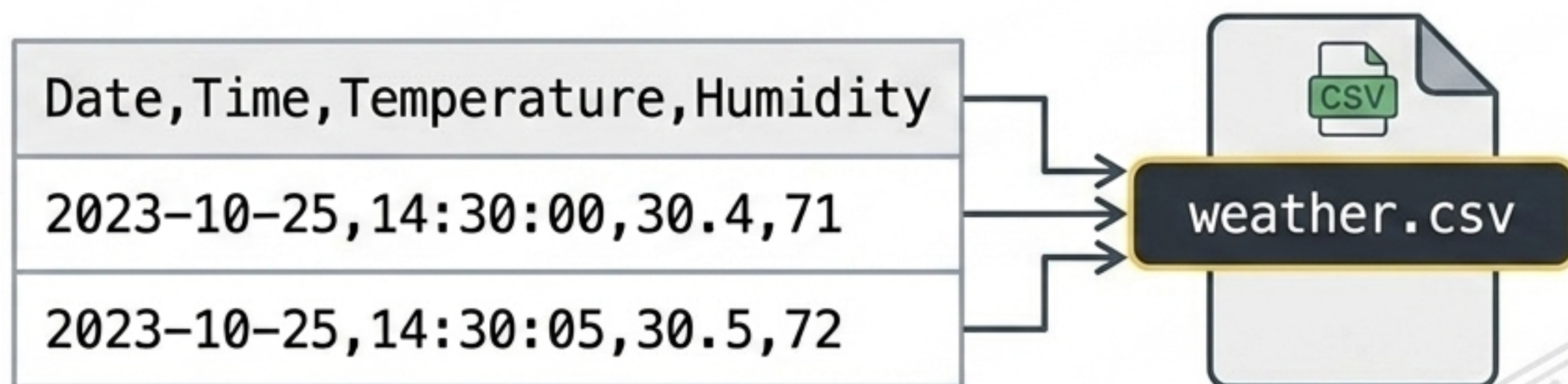
```
Temperature : 30.4 °C  
Humidity    : 71 %
```

Workshop:

- นำโค้ดบรรจลงในลูป (Loop) 
- ตั้งเวลาหน่วง (time.sleep(5)) เพื่อทดลองดึงข้อมูลสภาพอากาศแบบ Real-time ทุกๆ 5 วินาที 

การบันทึกประวัติข้อมูลลงไฟล์ (CSV Logging)

Why Log Data? ข้อมูลที่ดึงมาจะหายไปหากไม่บันทึก การบันทึกลงไฟล์ช่วยให้เราสามารถนำข้อมูลไปวิเคราะห์ย้อนหลังหรือสร้างกราฟได้

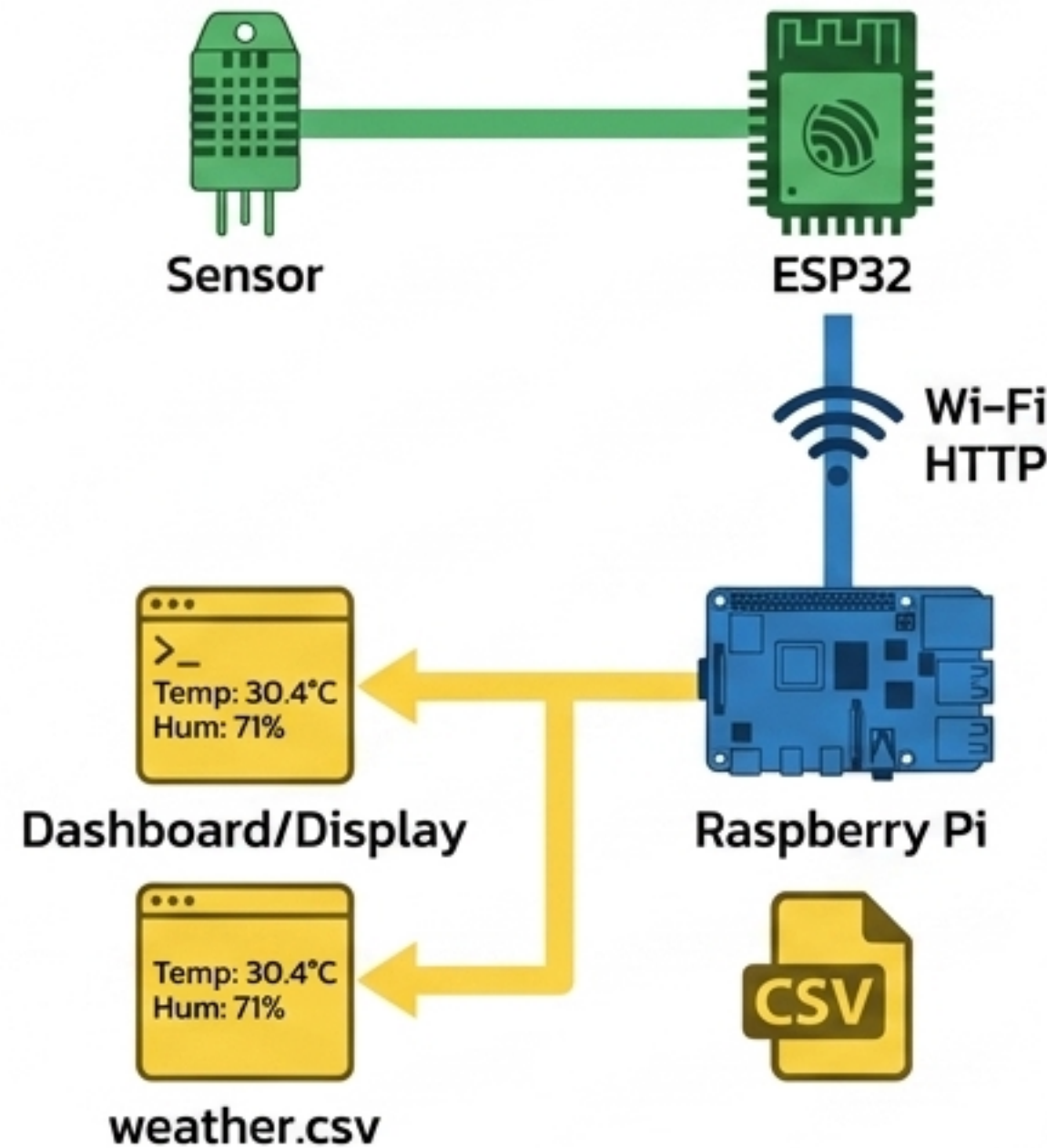


Workshop:

- เขียน Python เปิดไฟล์โหมด **Append ('a')**
- เพิ่ม Timestamp (วันที่และเวลา) เข้าไปกับชุดข้อมูล
- บันทึกข้อมูลแบบอัตโนมัติทุกครั้งที่อ่านค่าได้สำเร็จ

Mini Project: s:UU Weather Monitoring System

Data Journey



Mission Checklist / Lab 3

- ☐ 1. ESP32 เชื่อมต่อ Wi-Fi ท้องถิ่นสำเร็จ
- ☐ 2. ESP32 อ่านค่าจากเซ็นเซอร์ (DHT22/BME280) ได้ถูกต้อง
- ☐ 3. ESP32 สร้าง Web Server และส่งข้อมูลออกเป็น JSON
- ☐ 4. Raspberry Pi ดึงข้อมูลผ่าน HTTP (requests)
- ☐ 5. Raspberry Pi แสดงผลข้อมูลอุณหภูมิ/ความชื้นบนหน้าจอ
- ☐ 6. Raspberry Pi บันทึกข้อมูลลงไฟล์ weather.csv อย่างต่อเนื่อง

Note: นักเรียนแต่ละกลุ่มลงมือพัฒนาระบบนี้ให้สมบูรณ์ (เวลา 40 นาที)

สรุปความสำเร็จ: จากอุปกรณ์ Edge สู่ศูนย์กลางข้อมูล



Acquired Skills Matrix

Hardware	Network	Software
ควบคุม ESP32 และเซ็นเซอร์	เชื่อมต่อ Wi-Fi, สื่อสารด้วย HTTP Protocol	ออกแบบ REST API, สร้างโครงสร้าง JSON, ดึงและบันทึกไฟล์ (CSV) ด้วย Python

เป้าหมายต่อไป (ส่วนที่ 4)

ข้อมูลที่ถูกบันทึกไว้อย่างเป็นระบบบน Raspberry Pi พร้อมแล้วที่จะถูกนำไปแปลงเป็น Dashboard แสดงผลกราฟิกและระบบ IoT บน Web Application ที่สมบูรณ์แบบ